InsightAgent — API Documentation

Version: 1.0.0
Base URL: `http://localhost:8000`
Interactive Docs (Swagger UI): `http://localhost:8000/docs`
OpenAPI JSON: `http://localhost:8000/openapi.json`

Table of Contents

1. Authentication
2. Error Codes
3. Health
4. Employees
5. Products
6. Saved Queries
7. Analytics
8. Skills

Endpoint Overview

Method	Endpoint	Description	Auth Required
GET	`/`	Basic health check	No
GET	`/health`	Detailed health check with DB connectivity	No
GET	`/employees`	List all employees (paginated)	Yes
GET	`/employees/{employee_id}`	Get one employee by ID	Yes
GET	`/products`	List all products (paginated)	Yes
GET	`/products/{product_code}`	Get one product by code	Yes
POST	`/saved-queries`	Create a new saved query	Yes
GET	`/saved-queries`	List all saved queries (paginated)	Yes
GET	`/saved-queries/{query_id}`	Get one saved query by ID	Yes
PUT	`/saved-queries/{query_id}`	Partially update a saved query	Yes
DELETE	`/saved-queries/{query_id}`	Delete a saved query	Yes
GET	`/analytics/store-sales-summary`	Total revenue and order count by office	Yes
GET	`/analytics/product-ranking`	Products ranked by revenue	Yes
GET	`/analytics/employee-performance`	Sales KPIs per employee	Yes
GET	`/analytics/sales-trend`	Monthly revenue trend (optional year filter)	Yes
POST	`/analytics/analyze`	Natural language analysis via Qwen agent	Yes

Method	Endpoint	Description	Auth Required
GET	`/analytics/logs`	List past analysis run logs	Yes
GET	`/analytics/logs/{log_id}/turns`	Get per-turn LLM details for an analysis run	Yes
POST	`/skills/observe_schema`	Return full database schema	Yes
POST	`/skills/run_python_analysis`	Execute sandboxed Python analysis code	Yes

---

1. Authentication

All business endpoints require an **API key** passed as an HTTP header.

Header	Value
`X-API-Key`	Your configured API key (see `.env → API_KEY`)

Default key (development): `insightagent-secret-key`

Example:

```
curl http://localhost:8000/employees \
  -H "X-API-Key: insightagent-secret-key"
```

Authentication Behaviour

Scenario	HTTP Status	Response
Valid key	`2xx`	Normal response
Missing or wrong key	`401`	`{"error": "invalid_api_key", "message": "Invalid or missing X-API-Key header."}`
`API_KEY` env var not set	Any	Auth skipped (open access — development mode only)

> **Note:** The two health check endpoints (`GET /` and `GET /health`) do **not** require authentication.

---

2. Error Codes

All error responses are JSON objects. Standard HTTP status codes are used throughout.

HTTP Status Codes

Code	Meaning	Typical Cause
`200`	OK	Request succeeded
`201`	Created	Resource created (POST to `/saved-queries`)
`204`	No Content	Resource deleted successfully
`400`	Bad Request	Malformed request body or invalid parameter
`401`	Unauthorized	Missing or invalid `X-API-Key` header
`404`	Not Found	Requested resource does not exist
`422`	Unprocessable Entity	Request body, path parameter, or query parameter fails validation
`500`	Internal Server Error	Unexpected server-side error

Structured Error Response (401 / 404 / 500)

```
{
  "detail": {
    "error": "invalid_api_key",
    "message": "Invalid or missing X-API-Key header."
  }
}
```

422 Validation Error Response

FastAPI returns a structured list of all validation failures. Examples:

Invalid path parameter type (e.g. `GET /employees/not-a-number`):

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "employee_id"],
      "msg": "Input should be a valid integer, unable to parse string as an
integer",
      "input": "not-a-number"
    }
  ]
}
```

Query parameter out of range (e.g. `GET /products?limit=0`):

```
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": ["query", "limit"],
      "msg": "Input should be greater than or equal to 1",
      "input": "0"
    }
  ]
}
```

Missing required body field (e.g. `POST /saved-queries` without `title`):

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "title"],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

Skill-Specific Error Statuses

`/skills/run_python_analysis` returns HTTP `200` only on success. All error cases use standard HTTP status codes, with the error detail in `{"detail": {...}}`:

HTTP	`error_type`	`status`	Meaning
`200`	—	`success`	Code executed successfully
`403`	`SECURITY_VIOLATION`	`blocked`	AST guard rejected a forbidden import or call
`403`	`SQL_BLOCKED`	`failed`	SQL contained a non-SELECT statement
`408`	`EXECUTION_TIMEOUT`	`failed`	Code exceeded the 30-second timeout
`422`	`SYNTAX_ERROR`	`blocked`	Python syntax error detected before execution
`422`	`RUNTIME_ERROR`	`failed`	Code raised an exception during execution
`500`	`WORKER_ERROR`	`failed`	Internal subprocess worker error

---

3. Health

`GET /`

Basic health check. No authentication required.

Response `200`:

```
{
  "status": "ok",
  "service": "InsightAgent API",
  "version": "1.0.0",
  "docs": "/docs",
  "database": "enterprise_api"
}
```

`GET /health`

Detailed health check including database connectivity. No authentication required.

Response `200` (all healthy):

```
{
  "status": "ok",
  "db_connections": {
    "rw": "ok",
    "ro": "ok"
  }
}
```

Response `200` (degraded):

```
{
  "status": "degraded",
  "db_connections": {
    "rw": "ok",
    "ro": "error: (OperationalError) ..."
  }
}
```

Possible Errors: None — always returns `200`.

---

4. Employees

Read-only access to the `employees` table. All endpoints require `X-API-Key`.

`GET /employees`

List all employees with pagination.

Query Parameters:

Parameter	Type	Default	Constraints	Description
`skip`	integer	`0`	`≥ 0`	Number of records to skip
`limit`	integer	`100`	`1 – 500`	Maximum records to return

Example Request:

```
curl "http://localhost:8000/employees?skip=0&limit=10" \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:


```
{
  "total": 23,
  "items": [
    {
      "employeeNumber": 1002,
      "lastName": "Murphy",
      "firstName": "Diane",
      "extension": "x5800",
      "email": "dmurphy@classicmodelcars.com",
      "officeCode": "1",
      "reportsTo": null,
      "jobTitle": "President"
    }
  ]
}
```


Possible Errors: `401` Unauthorized · `422` Invalid query parameter · `500` Server error

---

`GET /employees/{employee_id}`

Get a single employee by their numeric ID.

Path Parameters:

Parameter	Type	Description
`employee_id`	integer	The `employeeNumber` of the employee

Example Request:

```
curl http://localhost:8000/employees/1002 \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
{
  "employeeNumber": 1002,
  "lastName": "Murphy",
  "firstName": "Diane",
  "extension": "x5800",
  "email": "dmurphy@classicmodelcars.com",
  "officeCode": "1",
  "reportsTo": null,
  "jobTitle": "President"
}
```

Response `404`:

```
{
  "detail": "Employee 9999 not found."
}
```

Possible Errors: `401` Unauthorized · `404` Employee not found · `422` Non-integer `employee_id` · `500` Server error

---

5. Products

Read-only access to the `products` table. All endpoints require `X-API-Key`.

`GET /products`

List all products with pagination.

Query Parameters:

Parameter	Type	Default	Constraints	Description
`skip`	integer	`0`	`≥ 0`	Number of records to skip
`limit`	integer	`100`	`1 – 500`	Maximum records to return

Example Request:

```
curl "http://localhost:8000/products?skip=0&limit=5" \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
{
  "total": 110,
  "items": [
    {
      "productCode": "S10_1678",
      "productName": "1969 Harley Davidson Ultimate Chopper",
      "productLine": "Motorcycles",
      "productScale": "1:10",
      "productVendor": "Min Lin Diecast",
      "productDescription": "This replica features...",
      "quantityInStock": 7933,
      "buyPrice": 48.81,
      "MSRP": 95.70
    }
  ]
}
```

Possible Errors: `401` Unauthorized · `422` Invalid query parameter · `500` Server error

---

`GET /products/{product_code}`

Get a single product by its code.

Path Parameters:

Parameter	Type	Description
`product_code`	string	The `productCode`, e.g. `S10_1678`

Example Request:

```
curl http://localhost:8000/products/S10_1678 \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
{
  "productCode": "S10_1678",
  "productName": "1969 Harley Davidson Ultimate Chopper",
  "productLine": "Motorcycles",
  "productScale": "1:10",
  "productVendor": "Min Lin Diecast",
  "productDescription": "This replica features...",
  "quantityInStock": 7933,
  "buyPrice": 48.81,
  "MSRP": 95.70
}
```

Response `404`:

```
{
  "detail": "Product INVALID_CODE not found."
}
```

Possible Errors: `401` Unauthorized · `404` Product not found · `500` Server error

---

6. Saved Queries

Full CRUD for the `saved_queries` table. All endpoints require `X-API-Key`.

`POST /saved-queries`

Create a new saved query.

Request Body:

Field	Type	Required	Description
`title`	string	✅	Short descriptive title
`natural_language_query`	string	✅	The original question in plain language
`generated_code`	string	❌	Python analysis code (if any)
`result_summary`	string	❌	Plain-text summary of the result

Example Request:

```
curl -X POST http://localhost:8000/saved-queries \
  -H "X-API-Key: insightagent-secret-key" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Revenue by Product Line",
    "natural_language_query": "What is the total revenue for each product
line?",
    "generated_code": "df = read_sql(\"SELECT productLine, SUM(quantityOrdered
* priceEach) AS revenue FROM orderdetails od JOIN products p ON od.productCode
= p.productCode GROUP BY productLine\")\nprint(df)",
    "result_summary": "Classic Cars had the highest revenue at $3.85M."
  }'
```

Response `201`:

```
{
  "id": 1,
  "title": "Revenue by Product Line",
  "natural_language_query": "What is the total revenue for each product line?",
  "generated_code": "df = read_sql(...)\nprint(df)",
  "result_summary": "Classic Cars had the highest revenue at $3.85M.",
  "created_at": "2025-04-13T10:00:00",
  "updated_at": "2025-04-13T10:00:00"
}
```

Possible Errors: `401` Unauthorized · `422` Validation error (missing required field) · `500` Server error

`GET /saved-queries`

List all saved queries with pagination.

Query Parameters:

Parameter	Type	Default	Constraints	Description
`skip`	integer	`0`	`≥ 0`	Number of records to skip
`limit`	integer	`100`	`1 – 500`	Maximum records to return

Example Request:

```
curl "http://localhost:8000/saved-queries?skip=0&limit=10" \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
{
  "total": 5,
  "items": [
    {
      "id": 1,
      "title": "Revenue by Product Line",
      "natural_language_query": "What is the total revenue for each product
line?",
      "generated_code": "...",
      "result_summary": "Classic Cars had the highest revenue.",
      "created_at": "2025-04-13T10:00:00",
      "updated_at": "2025-04-13T10:00:00"
    }
  ]
}
```

Possible Errors: `401` Unauthorized · `422` Invalid query parameter · `500` Server error

---

`GET /saved-queries/{query_id}`

Get a single saved query by ID.

Path Parameters:

Parameter	Type	Description
`query_id`	integer	The saved query ID

Example Request:

```
curl http://localhost:8000/saved-queries/1 \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`: *(same shape as a single item in the list above)*

Response `404`:

```
{
  "detail": "SavedQuery 999 not found."
}
```

Possible Errors: `401` Unauthorized · `404` Query not found · `422` Non-integer `query_id` · `500` Server error

---

`PUT /saved-queries/{query_id}`

Update one or more fields of a saved query. All fields are optional (partial update).

Path Parameters:

Parameter	Type	Description
`query_id`	integer	The saved query ID

Request Body (all optional):

Field	Type	Description
`title`	string	New title
`natural_language_query`	string	New question text
`generated_code`	string	New code
`result_summary`	string	New summary

Example Request:

```
curl -X PUT http://localhost:8000/saved-queries/1 \
  -H "X-API-Key: insightagent-secret-key" \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Title"}'
```

Response `200`: *(full updated object)*

Response `404`:

```
{
  "detail": "SavedQuery 999 not found."
}
```

Possible Errors: `401` Unauthorized · `404` Query not found · `422` Validation error · `500` Server error

---

`DELETE /saved-queries/{query_id}`

Delete a saved query by ID.

Path Parameters:

Parameter	Type	Description
`query_id`	integer	The saved query ID

Example Request:

```
curl -X DELETE http://localhost:8000/saved-queries/1 \
  -H "X-API-Key: insightagent-secret-key"
```

Response `204`: *(empty body)*

Response `404`:

```
{
  "detail": "SavedQuery 999 not found."
}
```

Possible Errors: `401` Unauthorized · `404` Query not found · `500` Server error

---

7. Analytics

Pre-built analytics queries and the Qwen LLM agent. All endpoints require `X-API-Key`.

`GET /analytics/store-sales-summary`

Total revenue and order count grouped by office/store.

Parameters: None

Example Request:

```
curl http://localhost:8000/analytics/store-sales-summary \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
[
  {
    "city": "San Francisco",
    "country": "USA",
    "total_revenue": 1234567.89,
    "order_count": 150
  }
]
```

Possible Errors: `401` Unauthorized · `500` Server error

`GET /analytics/product-ranking`

Products ranked by total revenue, descending.

Query Parameters:

Parameter	Type	Default	Constraints	Description
`limit`	integer	`20`	`1 – 100`	Number of top products to return

Example Request:

```
curl "http://localhost:8000/analytics/product-ranking?limit=5" \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
[
  {
    "productCode": "S18_3232",
    "productName": "1992 Ferrari 360 Spider red",
    "productLine": "Classic Cars",
    "total_revenue": 276839.98,
    "total_quantity": 1808
  }
]
```

Possible Errors: `401` Unauthorized · `422` Invalid `limit` parameter · `500` Server error

---

`GET /analytics/employee-performance`

Sales KPIs for every employee: total revenue, order count, and customer count.

Parameters: None

Example Request:

```
curl http://localhost:8000/analytics/employee-performance \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
[
  {
    "employeeNumber": 1370,
    "firstName": "Gerard",
    "lastName": "Hernandez",
    "jobTitle": "Sales Rep",
    "total_revenue": 1258577.81,
    "order_count": 14,
    "customer_count": 7
  }
]
```

Possible Errors: `401` Unauthorized · `500` Server error

`GET /analytics/sales-trend`

Monthly revenue trend across all years, or filtered to a single year.

Query Parameters:

Parameter	Type	Default	Description
`year`	integer	`null` (all years)	Filter results to a specific year, e.g. `2004`

Example Requests:

```
# All years
curl http://localhost:8000/analytics/sales-trend \
  -H "X-API-Key: insightagent-secret-key"

# Year 2004 only
curl "http://localhost:8000/analytics/sales-trend?year=2004" \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
[
  {
    "year": 2004,
    "month": 11,
    "monthly_revenue": 1029837.66,
    "order_count": 29
  }
]
```

Possible Errors: `401` Unauthorized · `422` Invalid `year` parameter · `500` Server error

---

`POST /analytics/analyze`

Submit a natural language question to the **Qwen self-loop agent**. The agent autonomously:

1. Calls `observe_schema` to understand the database structure
2. Writes and executes Python analysis code via `run_python_analysis`
3. Reads results, fixes errors, and iterates (up to 8 rounds)
4. Returns a structured answer via `final_answer`

The final result and generated code are automatically saved to `saved_queries`.

Request Body:

Field	Type	Required	Description
`question`	string	✅	The natural language question to analyze

Example Request:

```
curl -X POST http://localhost:8000/analytics/analyze \
  -H "X-API-Key: insightagent-secret-key" \
  -H "Content-Type: application/json" \
  -d '{"question": "Which product line generated the most revenue?"}'
```

Response `200`:

```
{
  "answer": "Classic Cars generated the highest revenue at $3,853,438.94,
followed by Vintage Cars at $1,903,150.84.",
  "iterations": 3,
  "generated_code": "import pandas as pd\ndf = read_sql('SELECT productLine,
SUM(quantityOrdered * priceEach) AS revenue
...')\nprint(df.to_csv(index=False))",
  "saved_query_id": 12,
  "log_id": 5,
  "tool_trace": ["observe_schema", "run_python_analysis", "final_answer"]
}
```

Response Fields:

Field	Type	Description
`answer`	string	The agent's conclusive answer
`iterations`	integer	Number of LLM rounds used
`generated_code`	string | null	Last Python code executed by the agent
`saved_query_id`	integer | null	ID of the auto-saved record in `saved_queries`
`log_id`	integer | null	ID in `analysis_logs` (used to retrieve per-turn details)
`tool_trace`	array of strings	Ordered list of tool calls, e.g. `["observe_schema", "run_python_analysis", "final_answer"]`

Possible Errors: `401` Unauthorized · `422` Validation error (missing `question`) · `500` LLM or server error

---

`GET /analytics/logs`

List all past analysis runs (from `analysis_logs`).

Query Parameters:

Parameter	Type	Default	Constraints	Description
`skip`	integer	`0`	`≥ 0`	Offset
`limit`	integer	`20`	`1 – 100`	Max records

Example Request:

```
curl http://localhost:8000/analytics/logs \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
[
  {
    "id": 5,
    "query_text": "Which product line generated the most revenue?",
    "status": "completed",
    "iterations": 3,
    "final_answer": "Classic Cars generated the highest revenue...",
    "created_at": "2025-04-13 10:00:00"
  }
]
```

Possible Errors: `401` Unauthorized · `422` Invalid query parameter · `500` Server error

---

`GET /analytics/logs/{log_id}/turns`

Retrieve every saved LLM turn for a specific analysis run, showing the agent's step-by-step reasoning.

Path Parameters:

Parameter	Type	Description
`log_id`	integer	The analysis log ID (from `POST /analytics/analyze → log_id`)

Example Request:

```
curl http://localhost:8000/analytics/logs/5/turns \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:


```
[
  {
    "id": 1,
    "iteration": 1,
    "tool_name": "observe_schema",
    "llm_content": "I need to understand the database structure first.",
    "tool_input": "{}",
    "tool_output": "{\"database\": \"enterprise_api\", \"tables\": [...]}",
    "created_at": "2025-04-13 10:00:01"
  },
  {
    "id": 2,
    "iteration": 2,
    "tool_name": "run_python_analysis",
    "llm_content": null,
    "tool_input": "{\"code\": \"import pandas as pd\\ndf = read_sql(...)\"}",
    "tool_output": "{\"status\": \"success\", \"stdout\":
\"productLine,revenue\\nClassic Cars,3853438.94\"}",
    "created_at": "2025-04-13 10:00:04"
  },
  {
    "id": 3,
    "iteration": 3,
    "tool_name": "final_answer",
    "llm_content": "Classic Cars generated the highest revenue at
$3,853,438.94.",
    "tool_input": null,
    "tool_output": null,
    "created_at": "2025-04-13 10:00:06"
  }
]
```


Response `404`:


```
{
  "detail": "Analysis log 999 not found."
}
```

Possible Errors: `401` Unauthorized · `404` Log not found · `422` Non-integer `log_id` · `500` Server error

---

8. Skills

Direct access to the two core analysis skills. Typically called by the agent internally, but also callable externally for testing or integration. All endpoints require `X-API-Key`.

`POST /skills/observe_schema`

Return the full schema of the `enterprise_api` database: all tables, columns, primary keys, and foreign keys. Uses the read-only `app_ro` account.

Request Body: Empty (no body required)

Example Request:

```
curl -X POST http://localhost:8000/skills/observe_schema \
  -H "X-API-Key: insightagent-secret-key"
```

Response `200`:

```
{
  "database": "enterprise_api",
  "tables": [
    {
      "table_name": "employees",
      "columns": [
        {
          "name": "employeeNumber",
          "declared_type": "INT",
          "nullable": false,
          "is_primary_key": true
        },
        {
          "name": "lastName",
          "declared_type": "VARCHAR(50)",
          "nullable": false,
          "is_primary_key": false
        }
      ],
      "foreign_keys": [
        {
          "from_column": "officeCode",
          "ref_table": "offices",
          "ref_column": "officeCode"
        },
        {
          "from_column": "reportsTo",
          "ref_table": "employees",
          "ref_column": "employeeNumber"
        }
      ]
    }
  ]
}
```


Response `500`:


```
{
  "detail": {
    "status": "failed",
    "error_type": "SCHEMA_READ_ERROR",
    "message": "Failed to inspect database schema."
  }
}
```

Possible Errors: `401` Unauthorized · `500` Schema read failed

`POST /skills/run_python_analysis`

Execute Python analysis code in an **isolated, sandboxed subprocess**. The sandbox provides:

- `read_sql(sql, params=(), max_rows=50000)` — execute a read-only SELECT query, returns a `pandas.DataFrame`
- `pandas` (as `pd`), `numpy` (as `np`), `math`, `statistics`, `datetime`, `re`
- All output **must** be produced via `print()`
- Maximum execution time: **30 seconds**

Security pipeline:

```
Input code
    │
    ▼
AST Guard  ──(blocked)──► SECURITY_VIOLATION error
    │ pass
    ▼
SQL Guard  ──(blocked)──► SQL_BLOCKED error
    │ pass
    ▼
Subprocess Worker (isolated process, app_ro DB account)
    │
    ▼
Structured JSON result
```

Forbidden imports/calls (AST-blocked before execution):

`os`, `sys`, `subprocess`, `socket`, `requests`, `pathlib`, `shutil`, `open()`, `eval()`, `exec()`, `compile()`, `__import__()`

Forbidden SQL (SQL guard): Any statement other than `SELECT` / `WITH` (e.g. `INSERT`, `UPDATE`, `DELETE`, `DROP`)

Request Body:

Field	Type	Required	Description
`code`	string	✅	Valid Python source code

Example Request:

```
curl -X POST http://localhost:8000/skills/run_python_analysis \
  -H "X-API-Key: insightagent-secret-key" \
  -H "Content-Type: application/json" \
  -d '{
    "code": "import pandas as pd\ndf = read_sql(\"SELECT productLine,
SUM(quantityOrdered * priceEach) AS revenue FROM orderdetails od JOIN products
p ON od.productCode = p.productCode GROUP BY productLine ORDER BY revenue
DESC\")\nprint(df.to_csv(index=False))"
  }'
```

Response `200` — Success:

```
{
  "status": "success",
  "stdout": "productLine,revenue\nClassic Cars,3853438.9400000002\nVintage
Cars,1903150.84\n...",
  "data_found": true,
  "summary": {
    "lines": 7,
    "chars": 210
  }
}
```

> `data_found` is `true` when `stdout` is non-empty, `false` when the query returned no rows.

Response `403` — Security Violation:

```
{
  "detail": {
    "status": "blocked",
    "error_type": "SECURITY_VIOLATION",
    "message": "Import of module 'os' is not allowed.",
    "hint": "Use only pandas, numpy, math, statistics, datetime, re."
  }
}
```

Response `403` — SQL Blocked:

```
{
  "detail": {
    "status": "failed",
    "error_type": "SQL_BLOCKED",
    "message": "Only SELECT / WITH statements are allowed."
  }
}
```

Response `422` — Syntax Error:

```
{
  "detail": {
    "status": "blocked",
    "error_type": "SYNTAX_ERROR",
    "message": "SyntaxError: invalid syntax (line 3)"
  }
}
```

Response `422` — Runtime Error:

```
{
  "detail": {
    "status": "failed",
    "error_type": "RUNTIME_ERROR",
    "message": "ZeroDivisionError: division by zero"
  }
}
```

Response `408` — Timeout:

```
{
  "detail": {
    "status": "failed",
    "error_type": "EXECUTION_TIMEOUT",
    "message": "Execution exceeded 30 seconds.",
    "hint": "Simplify the query or reduce the dataset size."
  }
}
```

Response `500` — Worker Error:

```
{
  "detail": {
    "status": "failed",
    "error_type": "WORKER_ERROR",
    "message": "Internal subprocess error."
  }
}
```

Possible Errors: `401` Unauthorized · `403` Security/SQL violation · `408` Timeout · `422` Syntax/runtime error or missing `code` field · `500` Worker error

---

Appendix: Agent Tool Trace Reference

The `tool_trace` field returned by `POST /analytics/analyze` lists every tool called by the agent in order:

Tool Name	Description
`observe_schema`	Agent inspected the database schema
`run_python_analysis`	Agent executed a Python code block
`final_answer`	Agent stopped the loop and returned the answer

Typical pattern:

```
observe_schema → run_python_analysis → run_python_analysis → final_answer
```

A `final_answer` call always appears last. To inspect the detailed input/output of each step, use `GET /analytics/logs/{log_id}/turns`.